

Assignment - 1

DA6400: Reinforcement Learning

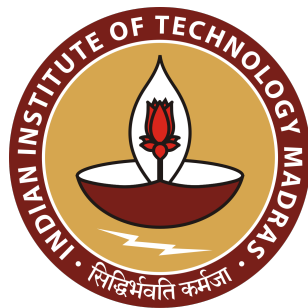
By

Ashmit Sinha and Shahid Mulla

ME21B037 and ME21B122

Course Instructor:

Prof. Balaram Ravindran



Indian Institute of Technology, Madras

March, 2025

Contents

1	Algorithms	1
1.1	Q-Learning and SARSA	1
1.1.1	Q-Learning	1
1.1.2	SARSA	1
1.2	Exploration Strategies: Softmax and Epsilon-Greedy	2
1.2.1	Softmax Exploration	2
1.2.2	Epsilon-Greedy Exploration	2
1.3	Conclusion	3
2	CartPole-v1	4
2.1	Introduction	4
2.1.1	State and Environment Details	4
2.1.2	Reward System	5
2.1.3	Transition Dynamics and Environment Settings	5
2.1.4	Training Objective	5
2.2	Implementation Specifics	5
2.2.1	State Space Discretization	5
2.2.2	Hyperparameter Optimization	6
2.3	Results	7
2.3.1	Experimental Results - Q learning with Softmax	7
2.3.2	Experimental Results - SARSA with Epsilon Greedy	8
2.3.3	Comparison: Q learning vs SARSA	10

3	MountainCar-v0	11
3.1	Introduction	11
3.1.1	State and Environment Details	11
3.1.2	Reward System	12
3.1.3	Transition Dynamics and Environment Settings	12
3.1.4	Training Objective	12
3.2	Implementation Specifics	12
3.2.1	State Space Discretization	12
3.2.2	Hyperparameter Optimization	13
3.3	Results	14
3.3.1	Experimental Results - Q-learning with Softmax	14
3.3.2	Experimental Results - SARSA with Epsilon Greedy	15
3.3.3	Comparison: Q learning vs SARSA	17
3.4	Reward Shaping	18
3.4.1	Energy-Based Reward Shaping	18
3.4.2	Modified Reward Function	18
3.4.3	Results of Reward Shaping	18
3.4.4	Effects of Reward Shaping	19
4	MiniGrid-Dynamic-Obstacles-5x5-v0 (Bonus)	20
4.1	Introduction	20
4.1.1	State and Environment Details	20
4.1.2	Reward System	21
4.1.3	Transition Dynamics and Environment Settings	21
4.1.4	Training Objective	21
4.2	Implementation specifics	22
4.2.1	Q-table Representation	22
4.3	Results	23
4.3.1	Experimental Results - Q-learning with Softmax	23
4.3.2	Experimental Results - SARSA with Epsilon Greedy	24

4.3.3	Comparison: Q-learning vs SARSA in Gridworld	24
5	Appendix - (Code Snippets)	26
5.1	CartPole-v1	26
5.2	MountainCar-v0	27
5.3	MiniGrid-Dynamic-Obstacles-5x5-v0	29
	Credits and Resources	30

Chapter 1

Algorithms

1.1 Q-LEARNING AND SARSA

Reinforcement learning algorithms optimize decision-making by learning an optimal policy through interaction with the environment. This section presents **Q-Learning** (off-policy) and **SARSA** (on-policy).

1.1.1 Q-Learning

Q-learning updates the action-value function using the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Algorithm 1 Q-Learning

```
Initialize  $Q(s, a)$  arbitrarily
Set number of episodes
for each episode do
  Initialize state  $s$ 
  while state  $s$  is not terminal or truncated do
    Choose action  $a$  from  $s$  using chosen policy (Softmax in our case)
    Take action  $a$ , observe reward  $r$  and next state  $s'$ 
    Update Q-value:
      
$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

    
$$s \leftarrow s'$$

  end while
end for
```

1.1.2 SARSA

SARSA is an on-policy algorithm that updates Q-values based on the selected action:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

Algorithm 2 SARSA

Initialize $Q(s, a)$ arbitrarily
Set number of episodes
for each episode **do**
 Initialize state s
 Choose action a from s using a policy
 while state s is not terminal or truncated **do**
 Take action a , observe reward r and next state s'
 Choose action a' from s' using chosen policy (Epsilon-greedy in our case)
 Update Q-value:
$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

 $s \leftarrow s', a \leftarrow a'$
 end while
end for

1.2 EXPLORATION STRATEGIES: SOFTMAX AND EPSILON-GREEDY

To balance **exploration** (discovering better actions) and **exploitation** (choosing the best-known action), we use **Softmax** and **Epsilon-Greedy** strategies.

1.2.1 Softmax Exploration

Softmax assigns probabilities to actions based on their Q-values:

$$P(a) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

Algorithm 3 Softmax Action Selection

Input: Q-values $Q(s, a)$, temperature τ
Compute probabilities:

$$P(a) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}}$$

Sample action a based on probabilities $P(a)$

1.2.2 Epsilon-Greedy Exploration

Epsilon-Greedy selects the best-known action most of the time but occasionally chooses a random action to explore:

$$a = \begin{cases} \text{random action} & \text{with probability } \epsilon \\ \arg \max_{a'} Q(s, a') & \text{with probability } 1 - \epsilon \end{cases}$$

Algorithm 4 Epsilon-Greedy Action Selection

Input: Q-values $Q(s, a)$, exploration rate ϵ

Generate random number $p \sim U(0, 1)$

if $p < \epsilon$ **then**

 Select a random action a

else

 Select $a = \arg \max_{a'} Q(s, a')$

end if

1.3 CONCLUSION

In this work, we use the following.

- **Q-Learning with Softmax Exploration**
- **SARSA with Epsilon-Greedy Exploration**

Through experimentation, we evaluated the effectiveness of both algorithms in learning an optimal policy for various environments. The results highlight the impact of exploration strategies on learning efficiency and overall performance.

Chapter 2

CartPole-v1

2.1 INTRODUCTION

The first setting for this assignment is the **CartPole** environment, as shown in Figure 1. The agent controls a cart that moves along a **one-dimensional track**, balancing a pole attached to it by a hinge as shown in Figure 2.1.



Figure 2.1: CartPole Environment

2.1.1 State and Environment Details

- **State Space:** The state is represented by a **4-dimensional** continuous vector:
 - Cart Position x : $-4.8 \leq x \leq 4.8$
 - Cart Velocity v : $-\infty \leq v \leq \infty$
 - Pole Angle θ : $-0.418 \text{ rad } (-24^\circ) \leq \theta \leq 0.418 \text{ rad } (24^\circ)$
 - Pole Angular Velocity ω : $-\infty \leq \omega \leq \infty$
- **Action Space:** The agent has **two available actions** at each time step:
 - **Push Left (0):** Applies force to the left.
 - **Push Right (1):** Applies force to the right.
- **Goal:** The goal of the agent is to balance the pole upright for as long as possible.
- **Start State:** All observations are assigned a uniformly random value in **(-0.05, 0.05)**

2.1.2 Reward System

- The agent receives a **reward of +1** for every time step it successfully balances the pole.
- The episode **ends** if either of the following conditions is met:
 - The angle of the pole $|\theta| > 12^\circ$ from vertical.
 - The cart moves too far from the center $|x| > 2.4$.
 - **Truncation:** Episode length is greater than 500

2.1.3 Transition Dynamics and Environment Settings

- **Physics-based transitions:** The next state is determined by simulating the cart’s movement using **Newtonian mechanics**.
- **Gravity:** The environment incorporates gravitational effects that cause the pole to fall unless countered by appropriate forces.
- **No stochasticity:** The transitions are **deterministic** given an action.

2.1.4 Training Objective

Our goal in this assignment is to train an **RL agent** to control the cart optimally and **maximize cumulative reward** while balancing the pole. We experiment with different algorithms, hyperparameters, and variations of the environment to determine the best-performing agent.

2.2 IMPLEMENTATION SPECIFICS

In our series of experiments, we first apply Q-learning with softmax exploration to the CartPole environment. The hyperparameters were tuned using **Weights & Biases** (WandB) to find optimal values. The state space was discretized to enable the application of tabular Q-learning, where the number of discretization bins was the same across all state variables but was treated as a hyperparameter.

2.2.1 State Space Discretization

Since the CartPole environment has a continuous state space, we discretized it to allow the application of tabular Q-learning and SARSA which make use of Q values for learning. Without discretization, the state-action space would be infinite, making it impractical to store Q-values for every possible state. By converting the continuous state space into discrete bins, we approximate the environment in a way that enables the algorithms to function effectively.

The state space consists of four variables: cart position, cart velocity, pole angle, and pole angular velocity. These were discretized using predefined bins, with the number of bins being a tunable hyperparameter. The binning strategy was chosen to reflect the varying nature of these variables:

- **Cart position:** Linearly discretized between $[-4.8, 4.8]$.

- **Cart velocity:** Non-linearly discretized using the hyperbolic tangent ($\tanh$) function between $[-5, 5]$ to capture variations more effectively in critical regions.
- **Pole angle:** Linearly discretized between $[-0.418, 0.418]$ radians.
- **Pole angular velocity:** Non-linearly discretized using $\tanh$ between $[-5, 5]$ to improve resolution near small values.

We implemented discretization using a `Discretizer` class, which maps continuous states to discrete bins using the `np.digitize()` function. Which basically maps the state to the index of the bin it belongs to, which later helps us in constructing the **Q table**.

2.2.2 Hyperparameter Optimization

We conducted a comprehensive hyperparameter search using WandB (~ 50 runs) to identify the optimal configuration for the agent following different policies. The following was the space of the hyperparameters that were tuned:

Q learning using Softmax

- Learning rate (α): $\{0.01, 0.05, 0.1\}$
- Initial temperature (τ_o): $\{1.0, 5.0, 10.0\}$
- Final temperature (τ_f): $\{0.1, 0.5, 1.0\}$
- Decay rate (τ_d): $\{0.99, 0.995, 0.999\}$
- State discretization granularity: $\{10, 20, 30, 50\}$

SARSA using Epsilon Greedy

- Learning rate (α): $\{0.01, 0.05, 0.1\}$
- Initial exploration probability (ϵ_o): $\{1.0, 0.5, 0.1\}$
- Final exploration probability (ϵ_f): $\{0.01, 0.05, 0.1\}$
- Decay rate (ϵ_d): $\{0.99, 0.995, 0.999\}$
- State discretization granularity: $\{10, 20, 30, 50\}$

2.3 RESULTS

The following section presents the results of two algorithms Q-learning with Softmax and SARSA with Epsilon Greedy applied to solving the CartPole-v1 environment.

2.3.1 Experimental Results - Q learning with Softmax

In the following subsection, we present the top 3 hyperparameter configurations found after performing a Weights & Biases (WandB) sweep. We conducted a Bayesian search over the hyperparameter space for a total of 5000 episodes for each combination and used regret minimization as a metric to pick the hyperparameter configurations.

Top 3 Best Hyperparameter Configurations

In our experiments with the CartPole environment, we plot the avg. rolling reward, we run the top 3 sets of hyperparameters for a total of 10,000 episodes averaged over 5 seed values to account for stochasticity.

Rank	Discretization Bins	Initial τ	Tau Decay	Final τ	Learning Rate
1	$20 \times 20 \times 20 \times 20$	10.0	0.995	1.0	0.1
2	$30 \times 30 \times 30 \times 30$	10.0	0.999	1.0	0.1
3	$20 \times 20 \times 20 \times 20$	10.0	0.999	0.1	0.1

Table 2.1: Top 3 hyperparameter configurations for Q-Learning with softmax action selection.

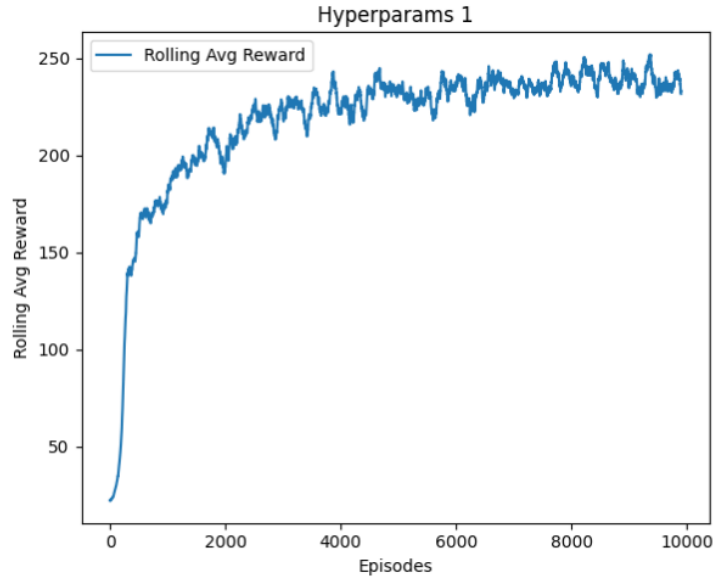


Figure 2.2: Hyper Parameters rank #1

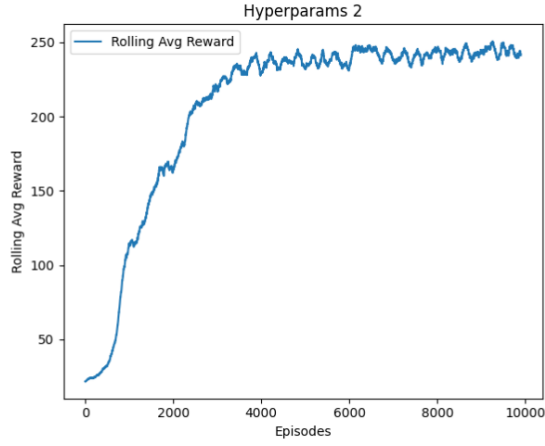


Figure 2.3: Hyper Parameters rank #2

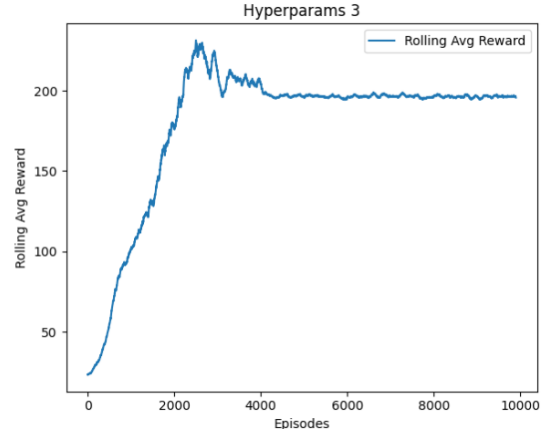


Figure 2.4: Hyper Parameters rank #3

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters:

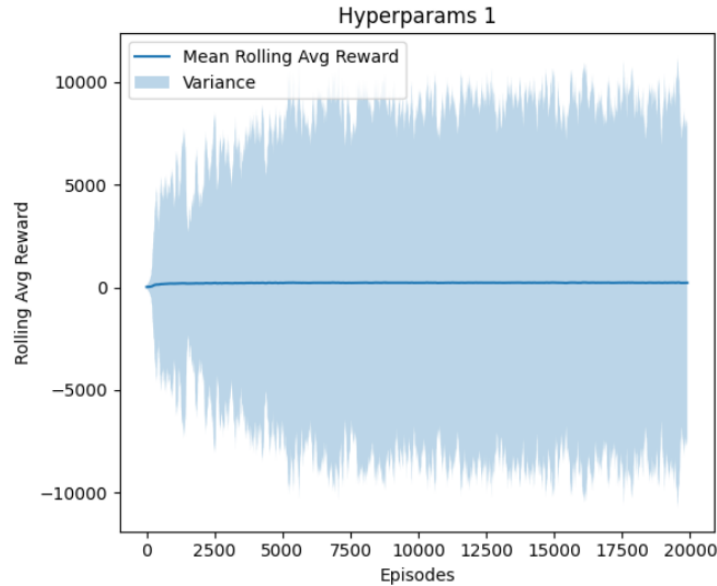


Figure 2.5: Hyper Parameters rank #1

2.3.2 Experimental Results - SARSA with Epsilon Greedy

Similar to Q-learning, we conducted a Weights & Biases (WandB) sweep for SARSA using epsilon-greedy exploration. We performed 50 runs, each lasting 5000 episodes, and selected the best hyperparameter configurations based on the highest achieved reward.

Top 3 Best Hyperparameter Configurations

In the experiments with the CartPole environment, to plot the avg. rolling reward, we run the top 3 set of hyperparameters for a total of 10,000 episodes averaged over 5 seed values to account for stochasticity.

Rank	Discretization Bins	Initial ϵ	Epsilon Decay	Final ϵ	Learning Rate
1	$20 \times 20 \times 20 \times 20$	0.5	0.999	0.05	0.05
2	$50 \times 50 \times 50 \times 50$	0.5	0.999	0.05	0.05
3	$50 \times 50 \times 50 \times 50$	1.0	0.999	0.1	0.1

Table 2.2: Top 3 hyperparameter configurations for SARSA ϵ -greedy action selection.

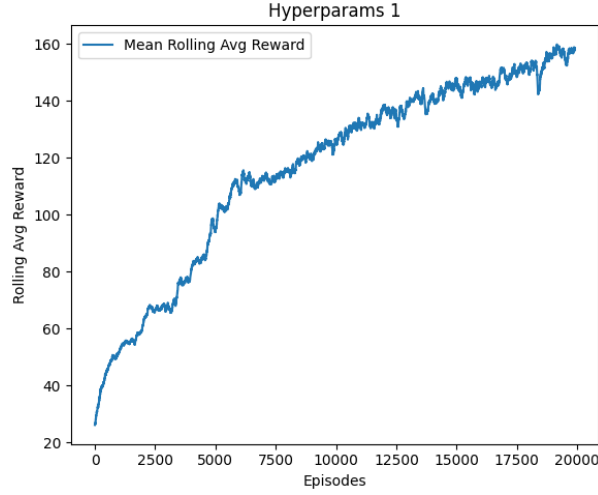


Figure 2.6: Hyper Parameters rank #1

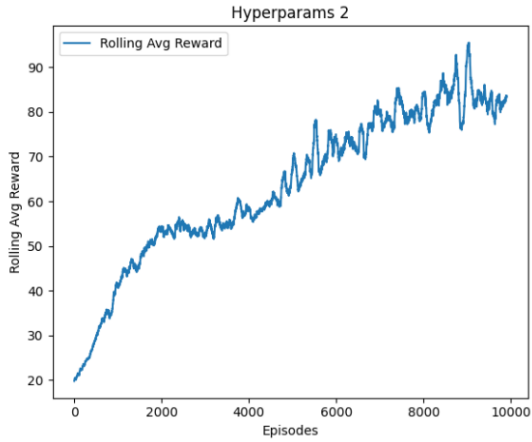


Figure 2.7: Hyper Parameters rank #2

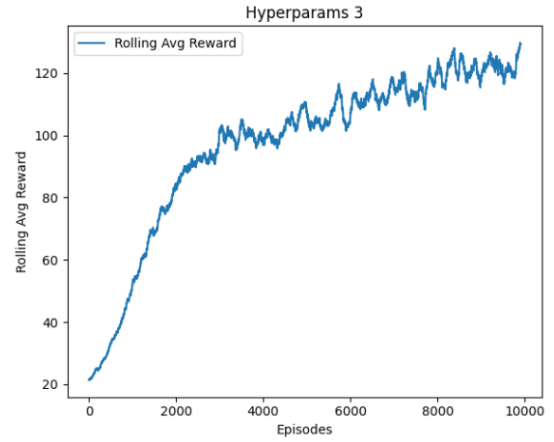


Figure 2.8: Hyper Parameters rank #3

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters for SARSA:

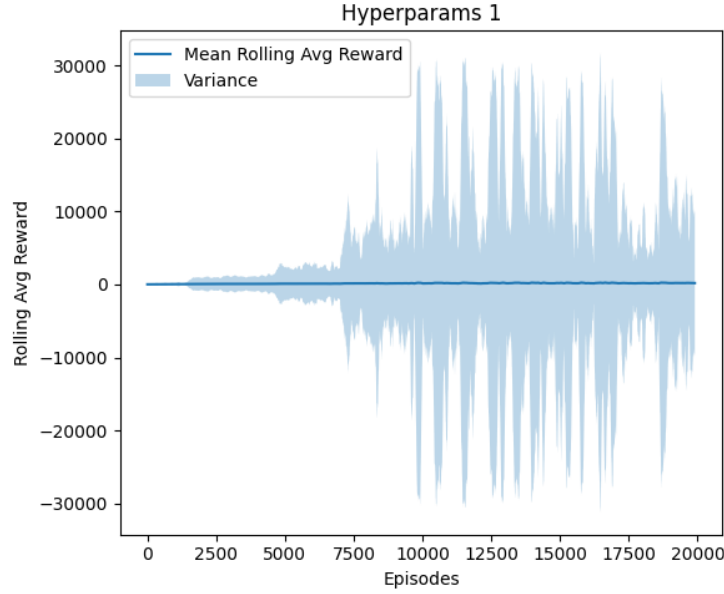


Figure 2.9: Hyper Parameters rank #1

2.3.3 Comparison: Q learning vs SARSA

Finally, we compare the performance of Q-learning and SARSA using their respective optimal hyperparameters.

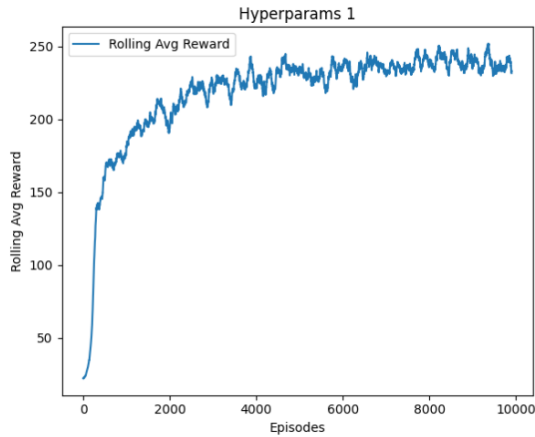


Figure 2.10: Rank #1 - Q learning

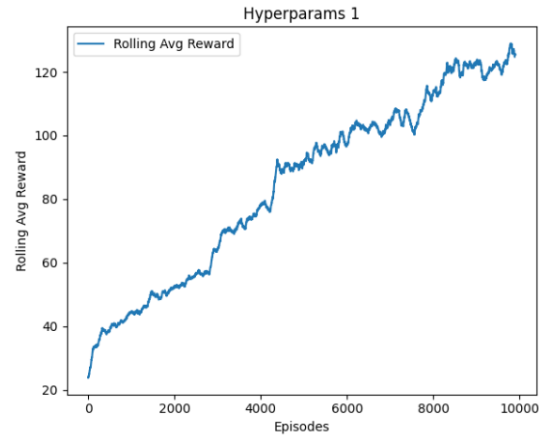


Figure 2.11: Rank #1 - SARSA

Key observations and inferences:

- **Q-learning achieves higher rewards and stabilizes faster** - The left plot shows Q-learning reaching a rolling average reward of around 250 and maintaining stability, while SARSA (right plot) improves steadily but reaches only about 120 by episode 10,000.
- **SARSA is more conservative in exploration** – The gradual increase in SARSA's performance indicates its on-policy nature leads to safer learning but potentially suboptimal policies, whereas Q-learning's off-policy updates allow for more aggressive improvements.

Chapter 3

MountainCar-v0

3.1 INTRODUCTION

The second environment for this assignment is the **MountainCar** environment, as shown in Figure 3.1. The agent controls a car that moves along a **one-dimensional track** between two hills shaped in sine curve. The objective is to build enough momentum to reach the flag at the top of the right hill.

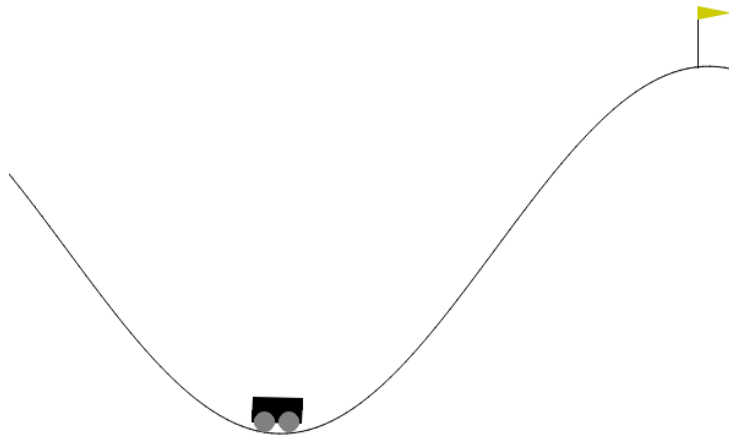


Figure 3.1: MountainCar Environment

3.1.1 State and Environment Details

- **State Space:** The state is represented by a **2-dimensional** continuous vector:
 - Car Position x : $-1.2 \leq x \leq 0.6$
 - Car Velocity v : $-0.07 \leq v \leq 0.07$
- **Action Space:** The agent has **three available actions** at each time step:
 - **Push Left (0):** Applies force to the left.
 - **No Push (1):** Applies no force.
 - **Push Right (2):** Applies force to the right.
- **Goal:** The goal of the agent is to reach the flag at $x = 0.5$ by gaining sufficient momentum.

- **Start State:** The car’s position is initialized randomly in **(-0.6, -0.4)**, and velocity is set to 0.

3.1.2 Reward System

- The agent receives a **reward of -1** for every time step until it reaches the goal.
- The episode **terminates** if either of the following conditions is met:
 - The car reaches the flag at $x \geq 0.5$.
 - **Truncation:** Episode length is greater than 200.

3.1.3 Transition Dynamics and Environment Settings

- **Physics-based transitions:** The next state is determined by simulating the cart’s movement using **Newtonian mechanics**.
- **Gravity Effect:** The environment incorporates a **gravitational pull**, requiring the agent to build momentum by moving back and forth.
- **No stochasticity:** The transitions are **deterministic** given an action.
- **Discretization:** The continuous state space can be discretized using binning, where position and velocity are mapped into discrete bins for tabular RL algorithms.

3.1.4 Training Objective

Our goal in this environment is to train an **RL agent** to control the car optimally and **maximize cumulative reward** by reaching the flag efficiently. We experiment with different algorithms, hyperparameters, and variations of the environment to determine the best-performing agent.

3.2 IMPLEMENTATION SPECIFICS

In our experiments, we applied Q-learning with softmax exploration and SARSA with epsilon-greedy exploration to the **MountainCar-v0** environment. We performed hyperparameter tuning using **Weights & Biases (WandB)** to optimize agent performance. Since the state space is continuous, we discretized it to enable the use of tabular Q-learning and SARSA, which rely on explicit Q-values. To further improve the performance of the agent, we tried to do **reward shaping** which will be discussed in the later sections.

3.2.1 State Space Discretization

MountainCar-v0 has a **two-dimensional continuous state space**, making direct tabular methods infeasible without discretization. We discretized the state space using a fixed number of bins, treating the bin size as a hyperparameter. This allowed the agent to generalize across similar states while keeping the Q-table manageable.

The state consists of two variables:

- **Car position (along x-axis):** Linearly discretized between $[-1.2, 0.6]$.

- **Car velocity:** Linearly discretized between $[-0.07, 0.07]$.

We implemented discretization using a `Discretizer` class, which maps continuous state values to discrete bins using the `np.digitize()` function. This transformation enabled us to construct a **Q-table** for learning.

3.2.2 Hyperparameter Optimization

We conducted extensive hyperparameter tuning (~ 50 runs) using WandB to determine the optimal settings for the agent’s learning process. Below are the hyperparameters explored for each method.

Q-learning with Softmax Exploration

- Learning rate (α): $\{0.01, 0.05, 0.1\}$
- Initial temperature (τ_o): $\{1.0, 5.0, 10.0\}$
- Final temperature (τ_f): $\{0.1, 0.5, 1.0\}$
- Decay rate (τ_d): $\{0.99, 0.995, 0.999\}$
- State discretization granularity: $\{10, 20, 30, 50\}$

SARSA with Epsilon-Greedy Exploration

- Learning rate (α): $\{0.01, 0.05, 0.1\}$
- Initial exploration probability (ϵ_o): $\{1.0, 0.5, 0.1\}$
- Final exploration probability (ϵ_f): $\{0.01, 0.05, 0.1\}$
- Decay rate (ϵ_d): $\{0.99, 0.995, 0.999\}$
- State discretization granularity: $\{10, 20, 30, 50\}$

3.3 RESULTS

This section presents the results of the two reinforcement learning algorithms, **Q-learning with Softmax** and **SARSA with Epsilon Greedy**, applied to solving the MountainCar-v0 environment.

3.3.1 Experimental Results - Q-learning with Softmax

We performed a Weights & Biases (WandB) sweep using a Bayesian search over the hyperparameter space. The search was conducted over 5000 episodes per configuration, using regret minimization to select the top 3 hyperparameter sets.

Top 3 Best Hyperparameter Configurations

For a fair evaluation, we ran the top 3 hyperparameter configurations for 50,000 episodes, averaging results over 3 random seeds to account for stochasticity.

Rank	Discretization Bins	Initial τ	Tau Decay	Final τ	Learning Rate
1	10×10	10.0	0.995	0.1	0.1
2	10×10	5.0	0.99	0.1	0.01
3	20×20	5.0	0.99	0.5	0.1

Table 3.1: Top 3 hyperparameter configurations for Q-learning with softmax action selection in MountainCar.

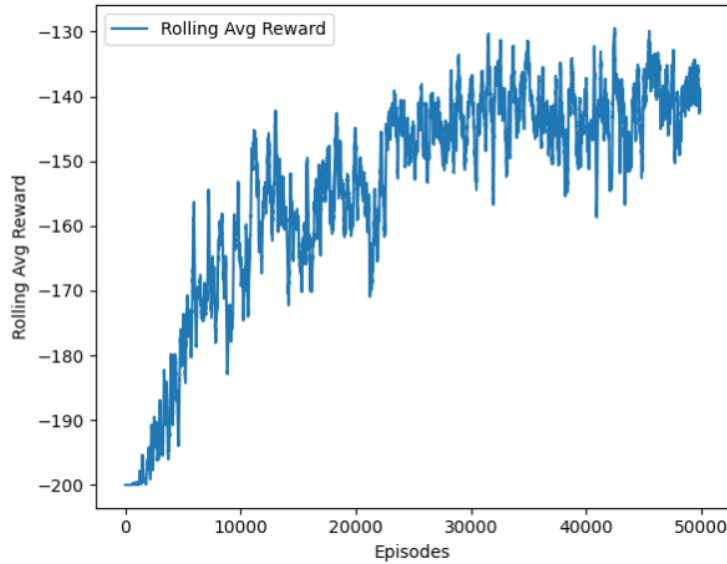


Figure 3.2: Rolling average reward for hyperparameter rank #1

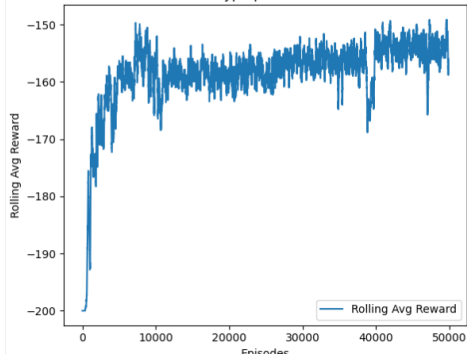


Figure 3.3: Hyperparameters rank #2

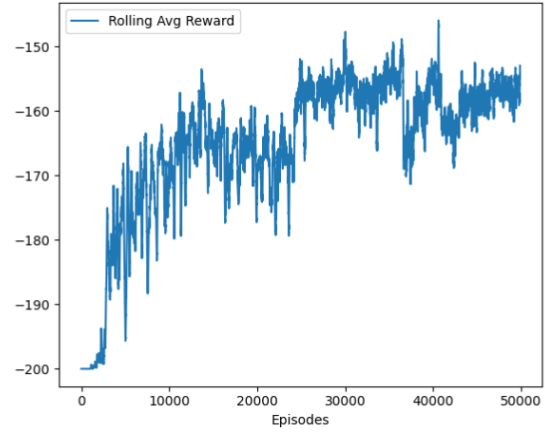


Figure 3.4: Hyperparameters rank #3

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters:

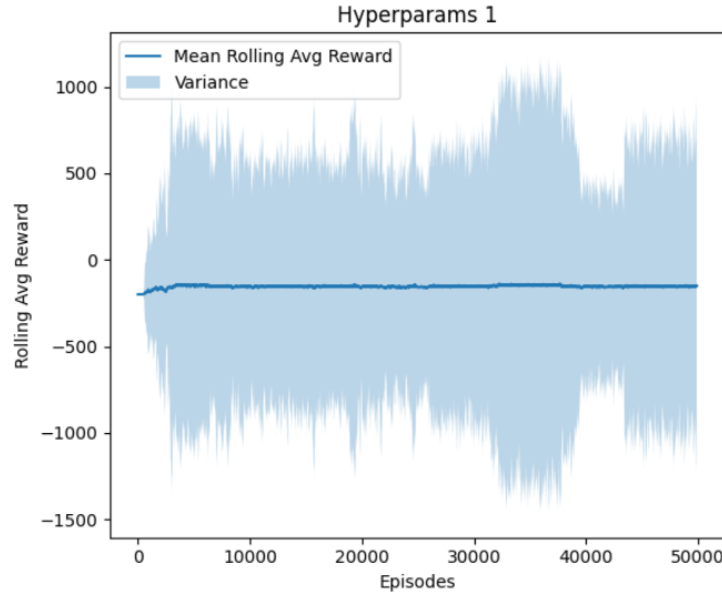


Figure 3.5: Hyper Parameters rank #1

3.3.2 Experimental Results - SARSA with Epsilon Greedy

A similar WandB sweep was conducted for SARSA, where we tested 50 hyperparameter sets, each running for 5000 episodes. The best configurations were chosen based on the highest cumulative reward.

Top 3 Best Hyperparameter Configurations

For evaluation, we ran the best 3 configurations for 50,000 episodes, averaging results over 3 random seeds.

Rank	Discretization Bins	Initial ϵ	Epsilon Decay	Final ϵ	Learning Rate
1	20×20	0.1	0.999	0.01	0.05
2	10×10	1.0	0.999	0.05	0.05
3	30×30	0.1	0.990	0.05	0.1

Table 3.2: Top 3 hyperparameter configurations for SARSA with ϵ -greedy action selection in MountainCar.

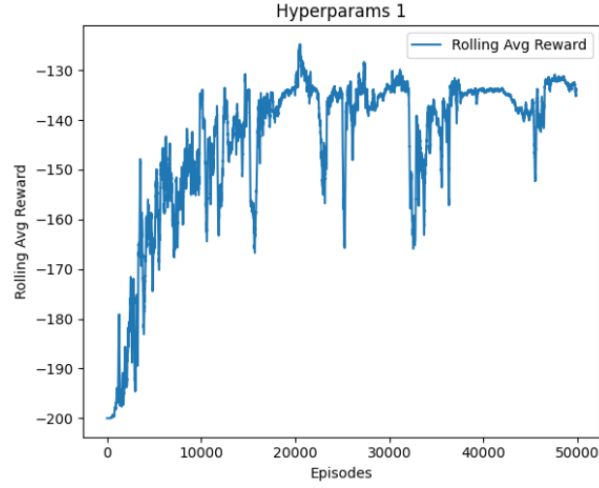


Figure 3.6: Rolling average reward for hyperparameter rank #1

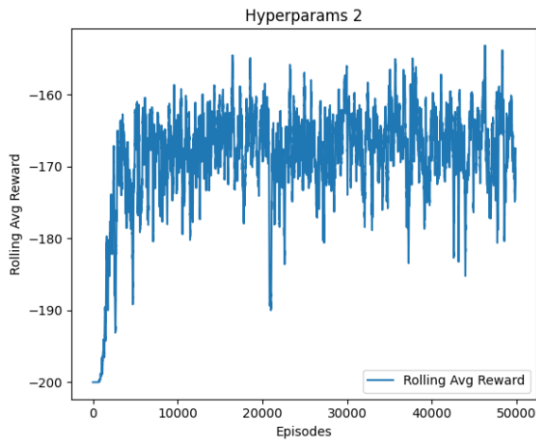


Figure 3.7: Hyperparameters rank #2

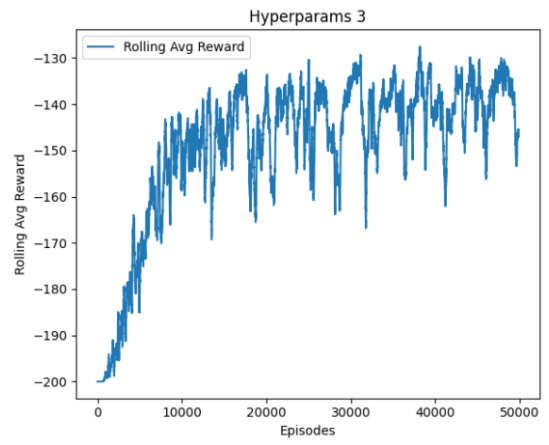


Figure 3.8: Hyperparameters rank #3

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters for SARSA:

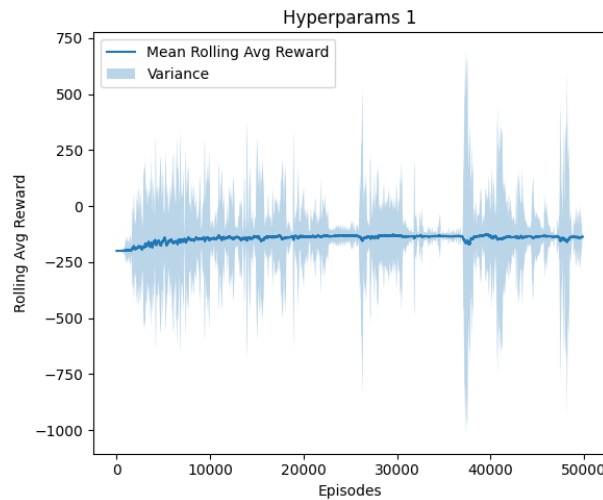


Figure 3.9: Hyper Parameters rank #1

3.3.3 Comparison: Q learning vs SARSA

Finally, we compare the performance of Q-learning and SARSA using their respective optimal hyperparameters.

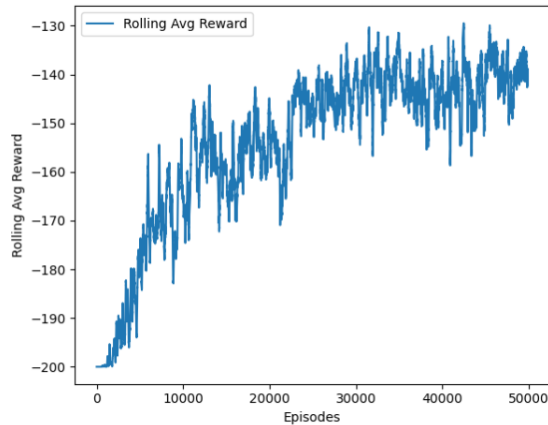


Figure 3.10: Rank #1 - Q learning

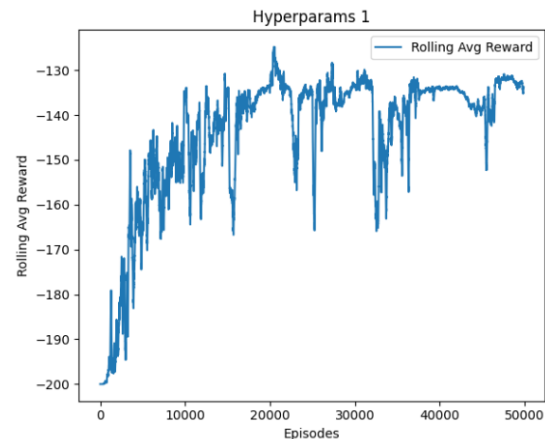


Figure 3.11: Rank #1 - SARSA

Key observations and inferences:

- Q-learning is more aggressive and fluctuates more but can reach a good policy faster.
- SARSA is more stable in early learning as it reaches the reward -140 by 10000 episodes, but suffers from occasional drops due to its cautious approach.
- MountainCar requires momentum-building strategies, where taking risks (exploration) helps escape the valley, Q-learning might be slightly better suited, but both methods eventually perform similarly.

3.4 REWARD SHAPING

Reinforcement Learning (RL) algorithms often struggle with sparse rewards, where meaningful feedback is provided only upon reaching the goal. In the case of the MountainCar-v0 environment, the agent receives a reward of -1 at each time step until it reaches the flag at the top. This sparse reward structure makes it difficult for the agent to explore efficiently and learn optimal policies.

In MountainCar-v0, the default reward function does not provide any incentive for gaining momentum or moving in the right direction, which can result in slow learning. To address this, we implement an **energy-based reward shaping** approach. Instead of rewarding position alone, we incorporate kinetic energy, encouraging the agent to build momentum efficiently.

3.4.1 Energy-Based Reward Shaping

Energy-based reward shaping leverages the principle that kinetic energy (KE) is proportional to velocity squared:

$$KE = \frac{1}{2}mv^2 \quad (3.1)$$

Since the mass (m) is constant in our environment, we simplify the reward function by directly incorporating velocity squared. This encourages the agent to move faster, gaining the necessary momentum to reach the goal.

3.4.2 Modified Reward Function

We modify the reward function as follows:

$$R(s, a) = -1 + 5v^2 \quad (3.2)$$

Additionally, we retain a large reward for successfully reaching the goal:

$$R_{\text{goal}} = 300, \quad \text{if } x \geq 0.5 \quad (3.3)$$

where x represents the position of the car, and v represents its velocity.

3.4.3 Results of Reward Shaping

We are implementing the new reward function with SARSA (ϵ -greedy) algorithm and comparing the performance with the same. The hyperparameters used in this comparison are the one's found to have least regret in the WandB sweep, that we named as Hyper Parameters rank #1. Similarly, for evaluation, we trained the agent with new reward for 50,000 episodes, averaging results over 3 random seeds.

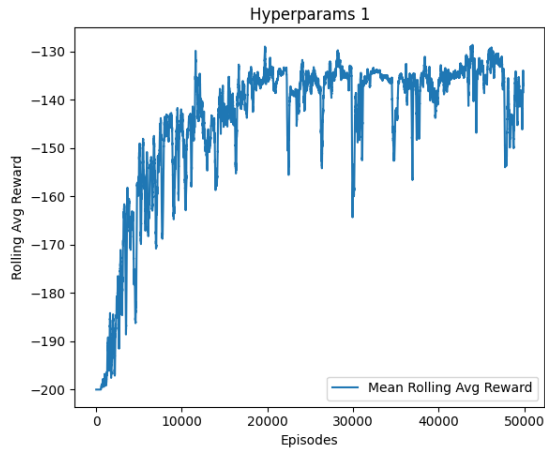


Figure 3.12: Rank #1 - SARSA (with reward shaping)

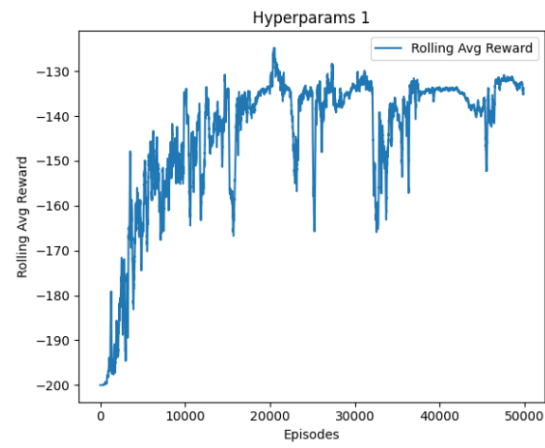


Figure 3.13: Rank #1 - SARSA (without reward shaping)

3.4.4 Effects of Reward Shaping

The energy-based reward shaping method is performing slightly better as compared to the standard reward function:

- Encourages the agent to gain momentum by rewarding velocity.
- Leads to slightly faster convergence compared to the standard reward function.
- Less number of occasional drops can be seen with the new reward function.

Mean and Variance Plots:

Below are the mean and variance plot for the best set of hyperparameters for SARSA with new reward function:

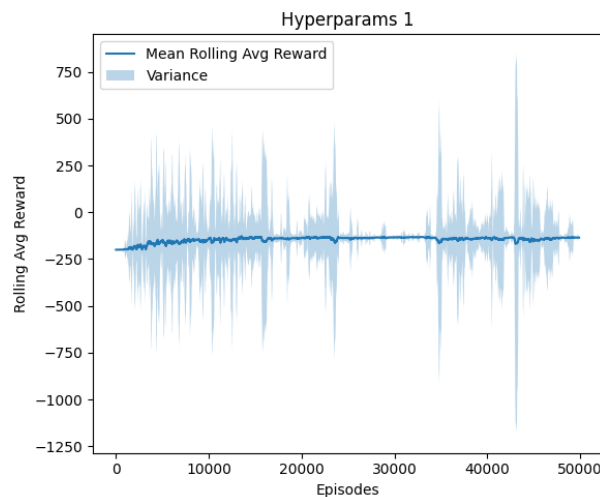


Figure 3.14: Hyper Parameters rank #1 with new reward

Chapter 4

MiniGrid-Dynamic-Obstacles-5x5-v0 (Bonus)

4.1 INTRODUCTION

The second setting for this assignment is the **MiniGrid-Dynamic-Obstacles-5x5-v0** environment, as shown in Figure 4.1. This environment consists of a **5x5 grid world** where the agent must navigate towards the goal while avoiding dynamic obstacles.

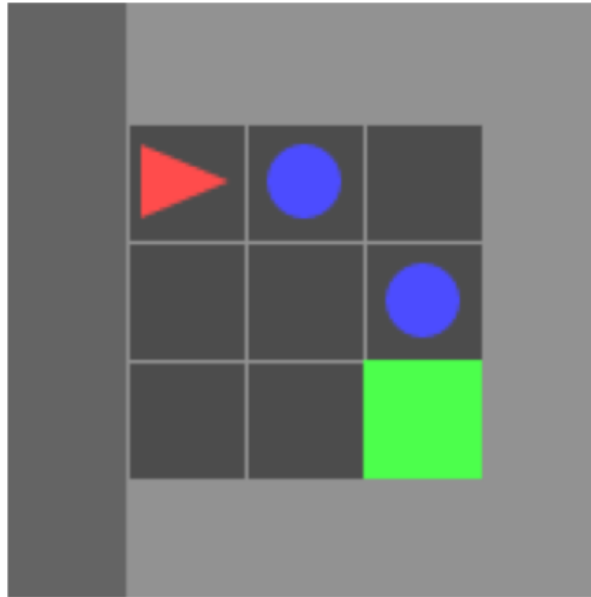


Figure 4.1: MiniGrid-Dynamic-Obstacles-5x5 Environment

4.1.1 State and Environment Details

- **State Space:** The state is represented as a partially observable **grid-based** environment. The agent receives the following information from the environment.
 - **Direction:** The direction of the agent (can take 4 discrete values)
 - **Image:** The visible part of the state space, size of 7x7x3. Each tile is encoded as a 3 dimensional tuple: (OBJECT_IDX, COLOR_IDX, STATE)

- **Mission:** A textual description of the task (not important in our case)
- **Action Space:** The agent has **6 discrete actions** but **only 3 are applicable for this particular env**:
 - **Move Forward (0):** Turn left
 - **Turn Left (1):** Turn right
 - **Turn Right (2):** Move Forward in the current direction
- **Goal:** The agent must navigate to the goal while avoiding collisions with dynamic obstacles.
- **Start State:** Predefined start state

4.1.2 Reward System

- The agent receives a **positive reward** for reaching the goal, computed as:

$$R = 1 - 0.9 \times \frac{\text{step_count}}{\text{max_steps}}$$

This encourages **efficient navigation** by rewarding shorter paths to the goal.

- The agent incurs a **penalty of -1** for colliding with a moving obstacle.
- 0 reward otherwise
- The episode **ends** if:
 - The agent successfully reaches the goal.
 - The agent collides with an obstacle.
 - The maximum number of steps is reached.

4.1.3 Transition Dynamics and Environment Settings

- **Dynamic obstacles:** The environment contains obstacles that move in predefined patterns, making navigation more challenging.
- **Deterministic agent actions:** The agent’s movements and rotations are deterministic.
- **Stochastic environment:** The obstacles move independently, introducing non-determinism.
- **Grid-based physics:** The environment follows a simple **grid-world** transition model.

4.1.4 Training Objective

The objective in this environment is to train an **RL agent** to navigate towards the goal while **avoiding collisions** with dynamic obstacles.

4.2 IMPLEMENTATION SPECIFICS

4.2.1 Q-table Representation

The Q-table size is determined as:

$$\text{Q-table size} = (\text{Action space}) \times (\text{Agent direction}) \times (\text{Possible state representations})$$

Action Space

Since our agent operates in a simple **5×5 Gridworld**, only the following actions are applicable:

- Turn Left
- Turn Right
- Move Forward

Thus, the action space size is **3**.

Agent Direction

The agent can face **4 possible directions** (Right, Down, Left, Up), making the direction space size **4**.

State Representation

Each **7×7 grid tile** is encoded as (OBJECT_IDX, COLOR_IDX), but only a subset of these values is relevant:

- **OBJECT_IDX**: 1 (empty), 2 (wall), 6 (ball), 8 (goal)
- **COLOR_IDX**: 0 (red), 2 (blue), 5 (grey)

This gives **12 possible tile states**.

Since the agent observes a **7×7** portion of the environment, the total number of possible states is:

$$7 \times 7 \times 12 = 588$$

Final Q-table Dimensions

Thus, the final Q-table has a size of:

$$3 \times 4 \times 588 = \mathbf{7,056}$$

This considers only **valid actions and environment constraints**, ensuring an efficient representation of state-action pairs.

4.3 RESULTS

This section presents the results of the two reinforcement learning algorithms, Q-learning with Softmax and SARSA with Epsilon Greedy, applied to solving the MiniGrid-Dynamic-Obstacles-5x5-v0.

4.3.1 Experimental Results - Q-learning with Softmax

We trained for a total of **1500 episodes**

Hyperparameters used (Found using Manual Search)

- Learning rate: $\alpha = 0.1$
- Initial exploration temperature: $\tau_0 = 1.0$
- Exploration decay: $\tau_{\text{decay}} = 0.995$
- Final exploration temperature: $\tau_{\text{final}} = 0.01$

Rolling rewards ((window size = 100), averaged over 3 seeds) vs episodes

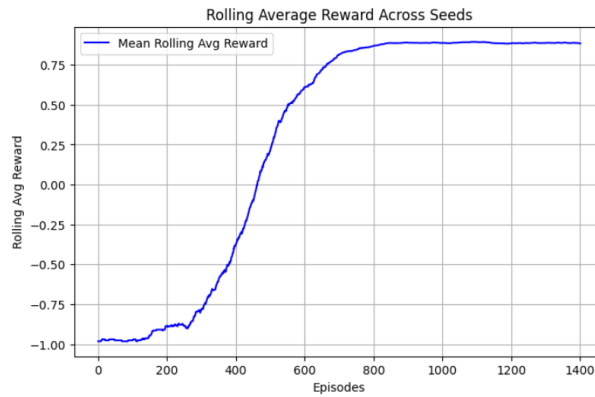


Figure 4.2: Q learning (with Softmax) for Gridworld

4.3.2 Experimental Results - SARSA with Epsilon Greedy

We trained for a total of **1500 episodes**

Hyperparameters used (Found using Manual Search)

- Learning rate: $\alpha = 0.05$
- Initial exploration temperature: $\epsilon_0 = 1.0$
- Exploration decay: $\epsilon_{\text{decay}} = 0.99$
- Final exploration temperature: $\epsilon_{\text{final}} = 0.01$

Rolling rewards ((window size = 100), averaged over 3 seeds) vs episodes

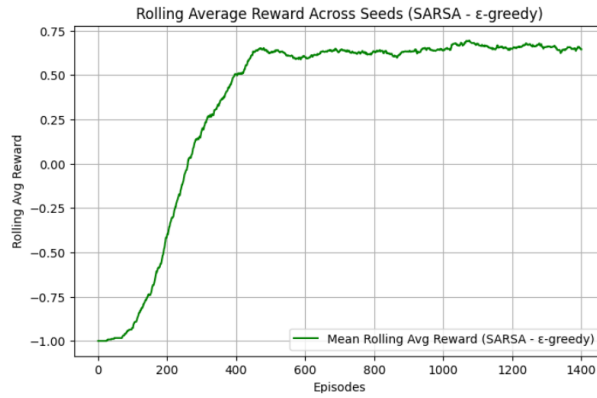


Figure 4.3: SARSA (with Epsilon Greedy) for Gridworld

4.3.3 Comparison: Q-learning vs SARSA in Gridworld

We compare the performance of Q-learning and SARSA using their respective optimal hyperparameters in the Gridworld environment.

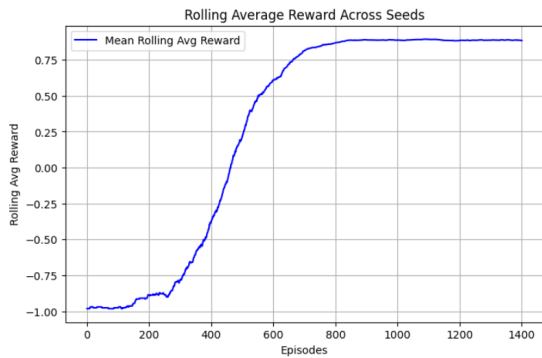


Figure 4.4: Q-learning in Gridworld

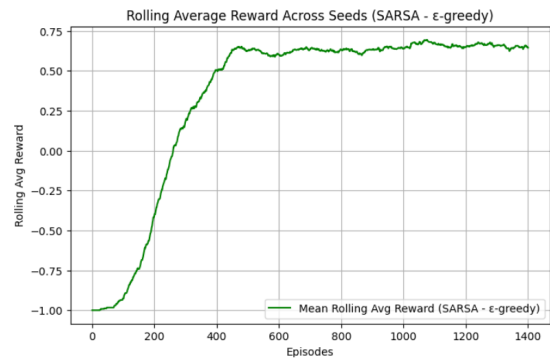


Figure 4.5: SARSA in Gridworld

Key observations and inferences:

- Q-learning converges to a higher final reward and exhibits more stability in rolling rewards.
- SARSA's rewards rise much faster in the initial episodes but plateau at a lower value compared to Q-learning.
- Due to its off-policy nature, Q-learning effectively explores and exploits the environment, leading to better long-term performance in Gridworld.

Chapter 5

Appendix - (Code Snippets)

5.1 CARTPOLE-V1

```
class Discretizer:
    def __init__(self, bins):
        self.bins = bins
        self.bin_sizes = [
            np.linspace(-4.8, 4.8, bins[0]), #cart position
            np.tanh(np.linspace(-5, 5, bins[1])), #cart velocity
            np.linspace(-0.418, 0.418, bins[2]), #pole angle (radians)
            np.tanh(np.linspace(-5, 5, bins[3])) #pole angular velocity
        ]

    def discretize(self, state):
        return tuple(np.digitize(s, self.bin_sizes[i]) for i, s in enumerate(state))
```

Figure 5.1: Discretizer Class

```
class Q_learning_cartpole_v1_softmax:
    def __init__(self, env, gamma, lr: float, initial_tau: float, tau_decay: float, final_tau: float, disc_bins: tuple, disc_factor: float = 0.99):
        self.env = env
        self.discretizer = Discretizer(bins=disc_bins)
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
        self.lr = lr
        self.tau = initial_tau
        self.tau_decay = tau_decay
        self.final_tau = final_tau
        self.disc_factor = disc_factor
        self.training_error = []

    def get_action(self, obs: np.ndarray) -> int:
        discrete_obs = self.discretizer.discretize(obs)
        q_vals_probab = self.q_values[discrete_obs] / self.tau
        exp_probab = np.exp(q_vals_probab) - np.max(q_vals_probab)
        action_probabilities = exp_probab / np.sum(exp_probab)
        return np.random.choice(np.arange(self.env.action_space.n), p=action_probabilities)

    def update(self, obs, action, reward, terminated, next_obs):
        discrete_obs = self.discretizer.discretize(obs)
        discrete_next_obs = self.discretizer.discretize(next_obs)
        future_q_value = (not terminated) * np.max(self.q_values[discrete_next_obs])
        temp_diff = reward + self.disc_factor * future_q_value - self.q_values[discrete_obs][action]
        self.q_values[discrete_obs][action] += self.lr * temp_diff
        self.training_error.append(temp_diff)

    def decay_tau(self):
        self.tau = max(self.final_tau, self.tau * self.tau_decay)
```

Figure 5.2: Q learning (with Softmax)

```

class SARSA.cartpole.v1.eps.greedy:
    def __init__(self, env: gym.Env, lr: float, initial_eps: float, eps_decay: float, final_eps: float, disc_factor: float = 0.99, discretization_bins=(20, 20, 20, 20)):
        self.env = env
        self.discretizer = Discretizer(bins=discretization_bins)
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
        self.lr = lr
        self.eps = initial_eps
        self.eps_decay = eps_decay
        self.final_eps = final_eps
        self.disc_factor = disc_factor
        self.training_error = []

    def get_action(self, obs: np.ndarray) -> int:
        discrete_obs = self.discretizer.discretize(obs)

        '''q_vals_probab = self.q_values[discrete_obs]/self.tau
        exp_probab = np.exp(q_vals_probab - np.max(q_vals_probab))
        action_probabilities = exp_probab / np.sum(exp_probab)
        return np.random.choice(np.arange(self.env.action_space.n), p=action_probabilities)'''

        if np.random.random() < self.eps:
            return self.env.action_space.sample()
        else:
            return int(np.argmax(self.q_values[discrete_obs]))

    def update(self, obs, action, reward, terminated, next_obs, next_action):
        discrete_obs = self.discretizer.discretize(obs)
        discrete_next_obs = self.discretizer.discretize(next_obs)

        future_q_value = self.q_values[discrete_next_obs][next_action] if not terminated else 0
        temp_diff = reward + self.disc_factor * future_q_value - self.q_values[discrete_obs][action]
        self.q_values[discrete_obs][action] += self.lr * temp_diff
        self.training_error.append(temp_diff)

    def decay_eps(self):
        self.eps = max(self.final_eps, self.eps*self.eps_decay)

```

Figure 5.3: SARSA (with Epsilon Greedy)

5.2 MOUNTAINCAR-V0

```

class Discretizer:
    def __init__(self, bins):
        self.bins = bins
        self.bin_sizes = [
            np.linspace(-1.2, 0.6, bins[0]), #position along x axis
            np.linspace(-0.07, 0.07, bins[1]) #velocity
        ]

    def discretize(self, state):
        return tuple(np.digitize(s, self.bin_sizes[i]) for i, s in enumerate(state))

```

Figure 5.4: Discretizer Class

```

class Q_learning.Mountain_car.v0.softmax:
    def __init__(self, env: gym.Env, lr: float, initial_tau: float, tau_decay: float, final_tau: float, disc_bins: tuple, disc_factor: float = 0.99):
        self.env = env
        self.discretizer = Discretizer(bins=disc_bins)
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
        self.lr = lr
        self.tau = initial_tau
        self.tau_decay = tau_decay
        self.final_tau = final_tau
        self.disc_factor = disc_factor
        self.training_error = []

    def get_action(self, obs: np.ndarray) -> int:
        discrete_obs = self.discretizer.discretize(obs)

        q_vals_probab = self.q_values[discrete_obs]/self.tau
        exp_probab = np.exp(q_vals_probab - np.max(q_vals_probab))
        action_probabilities = exp_probab / np.sum(exp_probab)
        return np.random.choice(np.arange(self.env.action_space.n), p=action_probabilities)

    def update(self, obs, action, reward, terminated, next_obs):
        discrete_obs = self.discretizer.discretize(obs)
        discrete_next_obs = self.discretizer.discretize(next_obs)

        future_q_value = (not terminated) * np.max(self.q_values[discrete_next_obs])
        temp_diff = reward + self.disc_factor * future_q_value - self.q_values[discrete_obs][action]
        self.q_values[discrete_obs][action] += self.lr * temp_diff
        self.training_error.append(temp_diff)

    def decay_tau(self):
        self.tau = max(self.final_tau, self.tau*self.tau_decay)

```

Figure 5.5: Q learning (with Softmax)

```

class SARSA_MountainCar_v0_eps_greedy:
    def __init__(self, env: gym.Env, lr: float, initial_eps: float, eps_decay: float, final_eps: float, discretization_bins: int, disc_factor: float = 0.99):
        self.env = env
        self.discretizer = Discretizer(bins=discretization_bins)
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
        self.lr = lr
        self.eps = initial_eps
        self.eps_decay = eps_decay
        self.final_eps = final_eps
        self.disc_factor = disc_factor
        self.training_error = []

    def get_action(self, obs: np.ndarray) -> int:
        discrete_obs = self.discretizer.discretize(obs)
        """q_vals probab = self.q_values[discrete_obs]/self.tau
        exp_probab = np.exp(q_vals probab - np.max(q_vals probab))
        action probabilities = exp_probab / np.sum(exp_probab)
        return np.random.choice(np.arange(self.env.action_space.n), p=action_probabilities)"""

        if np.random.random() < self.eps:
            return self.env.action_space.sample()
        else:
            return int(np.argmax(self.q_values[discrete_obs]))

    def update(self, obs, action, reward, terminated, next_obs, next_action):
        discrete_obs = self.discretizer.discretize(obs)
        discrete_next_obs = self.discretizer.discretize(next_obs)

        future_q_value = self.q_values[discrete_next_obs][next_action] if not terminated else 0
        temp_diff = reward + self.disc_factor * future_q_value - self.q_values[discrete_obs][action]
        self.q_values[discrete_obs][action] += self.lr * temp_diff
        self.training_error.append(temp_diff)

    def decay_eps(self):
        self.eps = max(self.final_eps, self.eps*self.eps_decay)

```

Figure 5.6: SARSA (with Epsilon Greedy)

```

def modified_reward(obs, reward, done):
    position, velocity = obs
    new_reward = -1

    new_reward += 5 * (velocity ** 2)

    if done and position >= 0.5:
        new_reward += 300

    return new_reward

```

Figure 5.7: Reward Shaping

5.3 MINIGRID-DYNAMIC-OBSTACLES-5X5-V0

```
class Q_learning_grid_world:
    def __init__(self, env, lr, initial_tau, tau_decay, final_tau, discount_factor = 0.99):
        self.env = env
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
        self.lr = lr
        self.tau = initial_tau
        self.tau_decay = tau_decay
        self.final_tau = final_tau
        self.discount_factor = discount_factor
        self.training_error = []

    def get_state_key(self, obs):
        image = tuple(obs["image"][:, :, :2].flatten())
        direction = obs["direction"]
        return (image, direction)

    def get_action(self, obs: np.ndarray) -> int:
        obs_key = self.get_state_key(obs)
        q_vals_probab = self.q_values[obs_key]/self.tau
        exp_probab = np.exp(q_vals_probab - np.max(q_vals_probab))
        action_probabilities = exp_probab/np.sum(exp_probab)
        return np.random.choice(np.arange(self.env.action_space.n), p=action_probabilities)

    def update(self, obs, action, reward, terminated, next_obs):
        obs_key = self.get_state_key(obs)
        next_obs_key = self.get_state_key(next_obs)
        future_q_value = (not terminated)*np.max(self.q_values[next_obs_key])
        temp_diff = reward + self.discount_factor*future_q_value - self.q_values[obs_key][action]
        self.q_values[obs_key][action] += self.lr*temp_diff
        self.training_error.append(temp_diff)

    def decay_tau(self):
        self.tau = max(self.final_tau, self.tau*self.tau_decay)
```

Figure 5.8: Q learning (with Softmax)

```
class SARSA_grid_world:
    def __init__(self, env, lr, initial_epsilon, epsilon_decay, final_epsilon, discount_factor=0.99):
        self.env = env
        self.q_values = defaultdict(lambda: np.zeros(env.action_space.n))
        self.lr = lr
        self.epsilon = initial_epsilon
        self.epsilon_decay = epsilon_decay
        self.final_epsilon = final_epsilon
        self.discount_factor = discount_factor
        self.training_error = []

    def get_state_key(self, obs):
        image = tuple(obs["image"][:, :, :2].flatten())
        direction = obs["direction"]
        return (image, direction)

    def get_action(self, obs: np.ndarray) -> int:
        obs_key = self.get_state_key(obs)
        if np.random.rand() < self.epsilon:
            return self.env.action_space.sample()
        else:
            return np.argmax(self.q_values[obs_key])

    def update(self, obs, action, reward, terminated, next_obs, next_action):
        obs_key = self.get_state_key(obs)
        next_obs_key = self.get_state_key(next_obs)
        future_q_value = (not terminated) * self.q_values[next_obs_key][next_action]
        temp_diff = reward + self.discount_factor * future_q_value - self.q_values[obs_key][action]
        self.q_values[obs_key][action] += self.lr * temp_diff
        self.training_error.append(temp_diff)

    def decay_epsilon(self):
        self.epsilon = max(self.final_epsilon, self.epsilon * self.epsilon_decay)
```

Figure 5.9: SARSA (with Epsilon Greedy)

Credits and Resources

- OpenAI GPT-4-turbo - For Latex boilerplate code